Comparison of methods for automatically predicting CVSS base vector

Sho Isogai
Shinshu University
Nagano, Japan
23w2009b@shinshu-u.ac.jp

Shinpei Ogata
Shinshu University
Nagano, Japan
ogata@cs.shinshu-u.ac.jp

Yutaro Kashiwa
Nara Institute of Science and Technology
Nara, Japan
yutaro.kashiwa@is.naist.jp

Satoshi Yazawa
Voice Research, Inc.
Tokyo, Japan
yazawa@voice-research.com

Kozo Okano
Shinshu University
Nagano, Japan
okano@cs.shinshu-u.ac.jp

Takao Okubo
Institute of Information Security
Kanagawa, Japan
okubo@iisec.ac.jp

Hironori Washizaki
Waseda University
Tokyo, Japan
washizaki@waseda.jp

2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC) | 979-8-3503-7696-8/24/$31.00 ©2024 IEEE | DOI: 10.1109/COMPSAC61105.2024.00140

***Abstract*—Common Vulnerability Scoring System (CVSS) is a standard method for quantifying the severity of software vulnerabilities. Security engineers can identify the underlying causes of vulnerabilities and develop countermeasures based on CVSS base vector and their related scores. The CVSS base vector is a set of metrics used to calculate the severity score of a vulnerability based on its intrinsic characteristics, such as "Attack Vector (AV)" and "Privilege Required (PR)". Security experts often take a few weeks to manually determine the CVSS base vector of a new vulnerability report. Consequently, the CVSS base vector is usually unknown for a period after the report is released, and thus delaying vulnerability countermeasures. Therefore, several methods for predicting CVSS base vector have been proposed. This paper compares the performance among methods that use BERT, multinomial logistic regression, or linear regression for automatically predicting CVSS base vector. The comparison results suggested that both BERT and MLR perform better than LR, with distinct advantages. BERT excels in understanding context, making it suitable for predicting general CVSS base vector, while MLR is effective for targeting specific attributes or severity levels. Consequently, these methods hold promise in aiding security engineers to promptly address vulnerabilities.**

***Index Terms*—CVE, CVSS, Vulnerability, BERT, Multinomial Logistic Regression, Linear Regression**

I. Introduction

Software vulnerabilities, leading to confidentiality, integrity, or availability failures, can be exploited by attackers, resulting in data tampering or denial of service. Upon discovery by security experts, vulnerability details, including CVE-ID, description, and severity, are cataloged in the National Vulnerability Database (NVD) [1]. Cisco Vulnerability Management highlights that a majority of attacks occur within two weeks of the CVE publication date [2], emphasizing the necessity for swift response.

Security engineers prioritize vulnerabilities [3], as prompt actions informed by analyzed information are crucial for attack prevention [4]. The Common Vulnerability Scoring System (CVSS) [5] aids in quantifying vulnerability severity. However, manually determining CVSS base vector for new reports often takes weeks [6], delaying countermeasures. To address these delays, methods for predicting CVSS base vector have emerged [6], [7], [8].

Unexplored questions include methods for enhancing accuracy, obtaining models more efficiently without sacrificing accuracy, and identifying specific characteristics of vulnerability descriptions hindering accurate CVSS base vector prediction by conventional methods. Therefore, we compare and analyze prediction results from CVSS-BERT [7], Multinomial Logistic Regression (MLR), and Linear Regression (LR) [6].

This paper presents comparative experiments on the aforementioned three methods. For this study, we replicated existing methods, CVSS-BERT [7] and LR [6], and introduced our implementation of the MLR method. The metric values of the CVSS base vector are denoted by labels corresponding to ordinal scale values. While the LR approach approximates continuous values to ordinal scale values, the MLR method predicts labels probabilistically. We presume these approaches to be fundamentally distinct.

All methods utilize the same datasets, obtained by dividing CVE data from 2018 to 2020 into training and testing sets. Vulnerability descriptions serve as input for all methods, which then output metric values. We assessed prediction performance and other characteristics by comparing predicted and actual metric values.

Our contributions can be summarized as follows:

(1) The BERT method is more accurate in predicting CVSS base vector and the MLR method is less likely to miss serious vulnerabilities. This enabled quicker CVSS base vector prediction with comparable performance.
(2) Tokens commonly associated with various CVSS metric values align with expert judgments. This maintains model interpretability.
(3) Some predictions were inaccurate, indicating the need for semantic sentence analysis to enhance accuracy.

The remainder of this paper is organized as follows. Section II explains BERT method [7], MLR method, and LR method [6]. Section III compares the prediction accuracy of BERT

2836-3795/24/$31.00 ©2024 IEEE
DOI 10.1109/COMPSAC61105.2024.00140

Authorized licensed use limited to: University of Education Winneba. Downloaded on September 16,2026 at 12:02:15 UTC from IEEE Xplore. Restrictions apply.

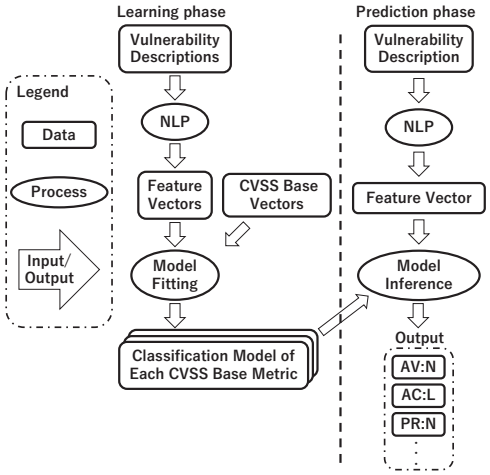


Fig. 1. Overview of the MLR method

method, MLR method, and LR method. Section IV summarizes related studies, and Section V concludes this paper.

II. Methods

This section outlines the CVSS base vector prediction methods employing BERT, MLR, or LR.

A. BERT method

This section describes the CVSS base vector prediction using BERT, proposed by Shahid et al. [7].

1) Step 1. Training BERT Classifiers: Train BERT classifiers for each CVSS metric (AC, AV, PR, UI, S, C, I, A). BERT is renowned for its performance in natural language processing.

2) Step 2. Prediction of CVSS Vector: Utilize trained BERT classifiers to predict metric values from textual vulnerability descriptions, forming an estimated CVSS vector.

3) Step 3. Calculation of Severity Score: Calculate the severity score using the predicted CVSS vector, comparing it with other vulnerabilities to assess severity.

4) Step 4. Ensuring Explainability: Ensure model predictions are explainable using a gradient-based input saliency method to identify relevant input words, aiding in understanding the model's predictions.

B. MLR method

This section outlines the CVSS base vector prediction method using MLR. Fig. 1 illustrates the MLR approach.

1) Step 1. NLP: Natural Language Processing: Create a feature vector from each vulnerability description using Bag of Words (BoW). Steps include:

1) Taking vulnerability descriptions as input data.
2) Extracting the set of all words from all the vulnerability descriptions, excluding stop words[1].
3) Generating a term frequency vector as a feature vector from each vulnerability description based on the word set.

Overall, this process is applied to vulnerability descriptions for both the learning and prediction phases.

2) Step 2. Model Fitting: Create MLR classification models for each CVSS base metric (AV, AC, PR, UI, S, C, I, and A). These models categorize vulnerability descriptions into the most probable value for each corresponding CVSS base metric using supervised learning.

3) Step 3. Model Inference: Predict the values of CVSS base metrics for given vulnerability descriptions using the trained MLR models. Utilize feature vectors from vulnerability descriptions to predict the most likely value for each CVSS base metric. Classes in MLR correspond to possible metric values.

C. LR method

This section describes the CVSS base vector prediction using LR, proposed by Elbaz et al. [6].

1) Step 1. Building the Linear Regression Model: Construct a linear regression model with training data, modeling the linear relationship between explanatory variables (features from descriptions) and response variables (CVSS vector elements).

2) Step 2. CVSS Vector Prediction: Predict the CVSS vector from newly disclosed vulnerability descriptions using the constructed linear regression model, nearly in real time.

3) Step 3. Evaluation Metrics: Evaluate predicted CVSS vector accuracy by comparing with actual CVSS vector from testing data, utilizing metrics like mean squared error (MSE) and coefficient of determination.

III. Evaluation

This section reports the evaluation of the BERT method, MLR method, and LR method.

A. Purpose

We evaluated the effectiveness of each method based on the following RQs.

- RQ 1 How do the prediction accuracies differ among the different methods?
- RQ 2 What input tokens have higher weights among the methods with high prediction accuracy?
- RQ 3 What are the characteristics of the vulnerability descriptions that reduce the prediction accuracy of each method?

We chose the method of Shahid et al. [7] as the BERT method because we believe that it is the most competitive and lightweight method to predict CVSS base vector. In addition, we chose the method of Elbaz et al. [6] as the LR method.

B. Data

We utilized NVD's annual vulnerability data for our study. We created two datasets from 45,926 vulnerabilities published from 2018 to 2020. One was used in the learning phase and the other was used in the prediction phase. These were split

[1] https://github.com/scikit-learn/scikit-learn/blob/82df48934eba1df9a1ed3be98aaace8eada59e6e/sklearn/feature_extraction/_stop_words.py

Authorized licensed use limited to: University of Education Winneba. Downloaded on September 16,2026 at 12:02:15 UTC from IEEE Xplore. Restrictions apply.

into two so that the metric values were evenly distributed in both datasets.

C. Implementation

We implemented the MLR method using Scikit-learn (version 1.0.2), a Python machine-learning library [9]. The MLR tool[2] is accessible online and can be executed on Google Colaboratory [10].

For the BERT method, we followed the implementation outlined in the paper by Shahid et al. [7]. Similarly, for the LR method, we implemented the approach described in the paper by Elbaz et al. [6].

D. Steps

The evaluator was one person understanding BERT, MLR, and LR methods and their tools. The evaluation environment was a computer equipped with an Intel(R) Core(TM) i9-7900X 3.30GHz with 64.00GB RAM and NVIDIA GeForce RTX 2080 Ti.

The steps of this evaluation are as follows.

1) The evaluator created classification models for each metric from vulnerability descriptions in the Train data using the BERT method, MLR method, or LR method. They measured the execution time.
2) The evaluator utilized classification models created using the BERT method, MLR method, or LR method to predict the value of each CVSS base metric for every vulnerability description in the test data.
3) The evaluator compared the accuracy, precision, recall, and F1 scores of BERT, MLR, and LR methods for predicting CVSS base metric values.
4) The evaluator identified the most relevant input tokens for determining cause-and-effect relationships in model predictions, focusing on the high-performing BERT and MLR methods based on classification model weights.
5) The evaluator analyzed instances where predicted CVSS base metrics differed from the actual values, examining reasons for these discrepancies.

E. Result

In summary, the MLR method achieved comparable scores to the BERT method and demonstrated fast prediction times. However, the LR method exhibited lower prediction performance compared to the other methods.

When utilizing the tool developed by Elbaz et al. [6], the number of vulnerabilities output by the tool's filter was reduced to 21,709.

Table I presents the performance of the BERT, MLR, and LR methods, with the bold part indicating the method with higher performance.

Tables II and III display the confusion matrices summarizing the prediction results for the 'AV' and 'PR' metrics using each method. In these tables, the vertical and horizontal axes represent the actual and predicted values, respectively. The gray areas indicate the number of vulnerabilities for which the predicted value matched the actual value.

We found a pattern in the discrepancy between actual and predicted metric values. The metric value of "Required Privileges (PR)" was most commonly predicted incorrectly.

Table IV shows the top 10 input tokens most relevant for "AV" and "PR" of BERT and MLR. Some tokens are common to both BERT and MLR methods. Unlike the BERT method, the MLR method includes proper nouns.

The evaluator measured the execution time from the start of processing raw vulnerability information to the creation of the classification model for both BERT and MLR methods. The BERT model took between 2497 and 2683 seconds to train, while the MLR model took between 4 and 6 seconds. The Attack Vector (AV) metric took the longest time, with the BERT method taking 2683 seconds, while the MLR method took only 6 seconds. In contrast, the AC metric took the least time, with the BERT method taking 2497 seconds and the MLR method taking 4 seconds.

F. Consideration

This section discusses research questions (RQs).

1) RQ 1. How do the prediction accuracies differ among the different methods?: Results indicate that the BERT method achieved accuracy rates ranging from 0.8379 to 0.9607 in predicting metric values, outperforming other methods. This highlights BERT's potential in CVSS base vector prediction. However, the MLR method demonstrated faster execution compared to BERT, suggesting the existence of accurate and lightweight approaches.

In contrast, the LR method exhibited lower prediction performance than other methods.

Fig. 2 illustrates the prediction results of the BERT and MLR methods for the Attack Vector (AV) metric. The colored areas represent the number of correctly predicted vulnerabilities. We particularly focus on the 762 vulnerabilities correctly predicted only by MLR.

The breakdown of these 762 vulnerabilities (shown on the right side of Fig. 3) reveals that over half, or 430 vulnerabilities, are classified as NETWORK, indicating the highest severity. On the other hand, the breakdown of the 849 vulnerabilities correctly predicted only by BERT (displayed on the left side of Figure 3) shows that approximately one-third are classified as NETWORK.

This suggests that MLR accurately predicts the category of "NETWORK," which represents the highest severity.

> Answer to RQ1:
> The BERT method is more accurate in predicting CVSS base vector and the MLR method is less likely to miss serious vulnerabilities.

2) RQ 2. What input tokens have higher weights among the methods with high prediction accuracy?: The tokens most commonly associated with different CVSS metric values are consistent with the reasoning of human cybersecurity

[2]https://zenodo.org/records/10780595

Authorized licensed use limited to: University of Education Winneba. Downloaded on September 16,2026 at 12:02:15 UTC from IEEE Xplore. Restrictions apply.

TABLE I
PERFORMANCE OF BERT, MLR AND LR METHODS

	BERT Method				MLR Method				LR Method			
Metrics	**Accuracy**	**Precision**	**Recall**	**F1 Score**	**Accuracy**	**Precision**	**Recall**	**F1 Score**	**Accuracy**	**Precision**	**Recall**	**F1 Score**
AV	**0.91**	**0.91**	**0.91**	**0.91**	0.90	0.90	**0.91**	0.90	0.77	0.88	0.77	0.79
AC	**0.96**	**0.96**	**0.96**	0.96	**0.96**	**0.96**	**0.96**	**0.97**	**0.96**	**0.96**	**0.96**	0.95
PR	**0.84**	**0.84**	**0.84**	**0.84**	0.83	0.82	0.83	0.82	0.77	0.77	0.77	0.76
UI	**0.93**	0.93	0.93	**0.93**	0.92	0.92	0.92	0.92	0.89	**0.99**	**0.98**	0.89
S	0.95	**0.96**	0.95	0.95	**0.96**	**0.96**	**0.96**	**0.96**	0.95	0.95	0.95	0.95
C	**0.87**	**0.87**	**0.87**	**0.87**	0.86	0.85	0.85	0.85	0.69	0.80	0.69	0.70
I	**0.87**	**0.87**	**0.87**	**0.87**	0.86	0.86	0.86	0.86	0.65	0.82	0.65	0.68
A	**0.89**	0.89	**0.89**	**0.89**	0.88	0.87	0.88	0.87	0.62	**0.93**	0.62	0.73

TABLE II
PREDICTION RESULTS FOR ATTACK VECTOR (AV)

Actual	**Predicted**	**BERT**	**MLR**	**LR**
Network	Network	16,332	16,482	15,238
	Adjacent Network	187	45	971
	Local	556	567	73
	Physical	26	7	11
Adjacent Network	Network	162	260	215
	Adjacent Network	397	281	192
	Local	18	36	44
	Physical	7	7	6
Local	Network	1,046	1,153	1,183
	Adjacent Network	43	9	2,225
	Local	3,883	3,815	1,258
	Physical	16	11	42
Physical	Network	57	102	36
	Adjacent Network	11	4	50
	Local	41	56	105
	Physical	181	128	60

TABLE III
PREDICTION RESULTS FOR PRIVILEGE REQUIRED (PR)

Actual	**Predicted**	**BERT**	**MLR**	**LR**
None	None	13,611	13,780	12,143
	Low	898	816	1,776
	High	186	99	22
Low	None	1,639	1,912	1,891
	Low	4,590	4,412	4,123
	High	258	163	122
High	None	380	529	284
	Low	296	432	919
	High	1,105	820	429

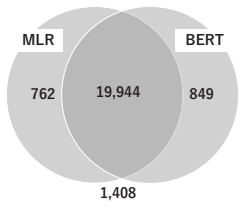


Fig. 2. The prediction results in AV

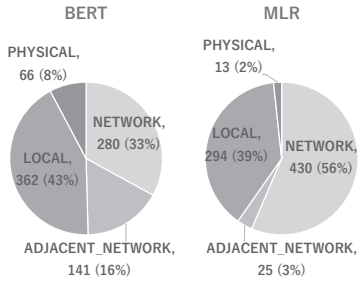


Fig. 3. Breakdown of vulnerabilities that were correctly predicted only

experts [6], [11]. For instance, the Attack Vector (AV) metric connects "physical" with Physical (P) and "local" with Local (L). Likewise, the Privileges Required (PR) metric links "administrators" with High (H) and "authenticated" with Low (L). These examples ensure the interpretability of machine learning models, enabling users to comprehend and clarify decisions [12]. This trait was confirmed in both BERT and MLR methods for all metrics.

Unlike the BERT method, MLR one incorporates proper nouns. For instance, the Attack Vector (AV) metric pairs "safari" with Network (N) and "intel" with Adjacent (A). Similarly, the Privilege Required (PR) metric associates "mcafee" with High (H). This arises from BoW-based NLP without dimensionality reduction. Conversely, CVSS-BERT pre-trained on extensive general text, might not encompass all proprietary or domain-specific terms. Hence, the influence of such terms on predictions is restricted.

Answer to RQ2:
The tokens most commonly associated with different CVSS metric values are consistent with the reasoning of human cybersecurity experts.

3) RQ 3. What are the characteristics of the vulnerability descriptions that reduce the prediction accuracy of each method?: We hypothesized that the inability to predict correctly might be due to missing information in the vulnerability description. Therefore, we investigated the number of words in the description.

Fig. 4 displays the distribution of the number of words in the test data. The left boxplot represents the distribution for all test data. The center boxplot shows the number of vulnerability descriptions correctly predicted by either the BERT or MLR method in the Attack Vector (AV) metric. The rightmost boxplot illustrates the results that could not be predicted correctly by either method in the AV metric. These boxplots exclude extreme outliers.

We observed no differences in the median or mean values among the boxplots. Therefore, based on this result, we conclude that missing information does not depend on the number of words.

Next, we discuss CVE-2018-8569 [13], a specific example of misclassification. The actual value of CVE-2018-8569 in AV is LOCAL, but the MLR method and BERT method predicted it to be NETWORK. This is thought to be because the content of the description is reminiscent of NETWORK.

We also identified a pattern in the discrepancy between actual and predicted metric values. Tables V present the top 5 patterns of discrepancies in metric values in descending

Authorized licensed use limited to: University of Education Winneba. Downloaded on September 16,2026 at 12:02:15 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
TOKENS MOST OFTEN ASSOCIATED TO SPECIFIC VALUES OF THE CVSS METRICS

		BERT Method	MLR Method
AV	**N**	'remote', 'xss', 'php', 'web', 'network', 'http', 'script', 'file', 'ur', 'site'	'edge', 'traversal', 'upload', 'scripting', 'inclusion', 'safari', 'csrf', 'php', 'remote', 'xss'
	A	'adjacent', 'attacker', 'cisco', 'network', 'via', 'route', 'same', 'access', 'intel', 'allows'	'radio', 'fi', 'wi', 'authenticated', 'devices', 'hyper', 'network', 'bluetooth', 'netgear', 'adjacent'
	L	'local', 'privilege', 'windows', 'privileges', 'file', 'kernel', 'elevation', 'remote', 'access', 'attacker'	'executes', '30102', '30452', '20063', 'crafted', 'xpdf', 'dll', 'artifex', 'logon', 'local'
	P	'physical', 'access', 'usb', 'device', 'allows', 'physically', 'intel', 'attacker', 'via', 'lock'	'smartphones', 'card', 'lock', 'proximate', 'screen', 'bypass', 'opensc', 'physically', 'usb', 'physical'
PR	**H**	'privileged', 'high', 'allows', 'attacker', 'admin', 'user', 'authentic', 'administrator', 'privileges', 'oracle'	'mcafee', 'hyper', 'administrators', 'administrative', 'authenticated', 'netgear', 'admin', 'administrator', 'privileged', 'high'
	L	'authenticated', 'local', 'user', 'users', 'allows', 'attacker', 'low', 'elevation', 'privileged', 'privilege'	'written', 'profile', 'additional', 'extended', 'e0506p09', 'jenkins', 'sharepoint', 'local', 'authenticated', 'low'
	N	'unauthenticated', 'remote', 'attackers', 'attacker', 'exploitation', 'code', 'corruption', 'vulnerability', 'network', 'successful'	'authenticated', 'local', 'privileged', 'nextcloud', 'privileges', 'freerdp', 'driver', 'configure', 'atlassian', 'sharepoint'

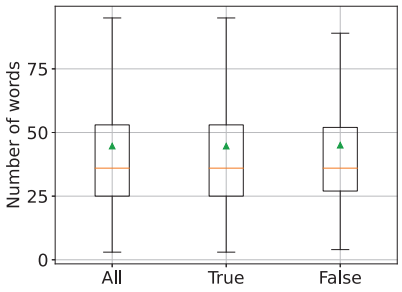


Fig. 4. Distribution of the number of words in vulnerability descriptions used in test data.

order of frequency. The vertical axis represents the top 5 patterns (P1 to P5), with gray cells indicating inconsistent metric values. The most common pattern among erroneously predicted CVSS base vector was that only the metric value of "Required Privileges (PR)" was incorrect. For example, Table VI illustrates an inconsistency in predicting the gray-shaded metric value "PR:H," which contradicted reality. This metric assesses the needed privilege level for the vulnerability-triggering action, and words in the gray zone of the table are notably pertinent. Although the term tied to privilege was "authenticated," the repetition of "administrator" in impact descriptions likely led to the misprediction of high "Privilege Required (PR)." Enhancements could involve extracting features solely from privilege-related sentences, necessitating a method to differentiate words' relevance and meaning, and separating privilege-related sentences from vulnerability impact descriptions.

Answer to RQ3:
At first glance, there are words in the sentences that are related to the metric values and lead to false predictions. To improve this problem, a method that analyzes the meaning of sentences can be introduced. Or we may need to refer to information other than the CVE.

TABLE V
PATTERNS IN WHICH PREDICTED METRIC VALUES WERE INCORRECT (BERT&MLR)

BERT

Pattern	AV	AC	PR	UI	S	C	I	A	Amount
P1	○	○	×	○	○	○	○	○	1,808
P2	○	○	○	○	○	×	○	○	918
P3	○	○	○	○	○	○	○	×	842
P4	×	○	○	○	○	○	○	○	796
P5	○	○	○	○	○	○	×	○	635

MLR

Pattern	AV	AC	PR	UI	S	C	I	A	Amount
P1	○	○	×	○	○	○	○	○	1,919
P2	○	○	○	○	○	×	○	○	749
P3	×	○	○	○	○	○	○	○	734
P4	○	○	○	×	○	○	○	○	687
P5	○	○	○	○	○	○	×	○	589

TABLE VI
AN EXAMPLE OF A DISCREPANCY ONLY FOR THE METRIC "PR" (CVE-2019-0018)

CVE-ID	CVE-2019-0018							
Actual	AV:N	AC:L	PR:L	UI:R	S:C	C:L	I:L	A:N
Predicted	AV:N	AC:L	PR:H	UI:R	S:C	C:L	I:L	A:N
Description	A persistent cross-site scripting (XSS) vulnerability in the file upload menu of Juniper ATP may allow an authenticated user to inject arbitrary scripts and steal sensitive data and credentials from a web administration session, possibly tricking a follow-on administrative user to perform administrative actions on the device. This problem affects Juniper ATP 5.0 versions prior to 5.0.3.							

IV. RELATED WORK

Security experts typically assess CVSS base scores manually based on their expertise, which can be time-consuming [11]. To address this, automated methods for predicting CVSS base scores have been explored [6], [8]. Khazaei et al. [8] proposed using Support Vector Machine (SVM), Random Forest Algorithm, and fuzzy theory [14] for this purpose. However, their focus was on vulnerability prioritization, which differs from the MLR method's aim of predicting CVSS base vector.

Meanwhile, Elbaz et al. [6] emphasize the importance of predicting both CVSS base scores and vectors. They propose a linear regression-based method to predict CVSS base metric

Authorized licensed use limited to: University of Education Winneba. Downloaded on September 16,2026 at 12:02:15 UTC from IEEE Xplore. Restrictions apply.

values, aiming to identify vulnerabilities' causes and suggest countermeasures. In contrast, our MLR method treats CVSS base metric value prediction as a classification problem rather than a linear one. Evaluation results indicate that both BERT and MLR methods outperform Elbaz et al.'s approach in terms of accuracy and efficiency in predicting CVSS base vector.

Shahid et al. [7] and Costa et al. [15] suggest leveraging BERT (Bidirectional Encoder Representations from Transformers) for CVSS base score prediction from vulnerability descriptions. While deep learning is effective, it can be time-consuming to train [15]. Turc et al. introduced BERT-small, a computationally less expensive variant with fewer parameters and faster training [16]. However, rebuilding the classification model may still require more time compared to the MLR method and LR method.

Han et al. [17] introduce a method to predict qualitative CVSS severity labels (Low, Medium, High, Critical) from vulnerability descriptions using a Convolutional Neural Network (CNN). They employ word embeddings trained with a continuous skip-gram model. Although effective, the CNN model lacks transparency as it does not offer explanations for its predictions.

Moura et al. [18] delve into the experiences of researchers from SIDN Labs and TU Delft regarding the discovery of DNS vulnerabilities and the subsequent disclosure process. They discuss the challenges involved, including the time-consuming nature and emotional toll of public disclosure. Ethical concerns and potential repercussions are also explored, alongside suggested improvements based on their insights. We propose that the MLR method could improve the information disclosure process by streamlining the evaluation of explanations.

V. Conclusion and Future Work

This study has introduced and assessed a methodology for the automated prediction of CVSS base vector using BERT, MLR, or LR. The evaluation results demonstrate that both the BERT and MLR approaches outperform the LR method, offering distinct advantages. BERT excels in comprehensively understanding context, making it suitable for predicting general CVSS base vector, especially when considering diverse contexts and nuances. Conversely, MLR is effective for targeting specific attributes or severity levels, such as identifying highly severe vulnerabilities for focused mitigation efforts. Consequently, these methods hold promise in assisting security engineers in promptly addressing vulnerabilities.

In future research endeavors, several challenges regarding BERT and MLR methodologies will need to be addressed. Primarily, the current approach still exhibits inaccuracies in predicting numerous CVSS base vector where one metric's value is incorrect. Detailed analysis of these inaccuracies is planned to identify problematic keywords and rectify the issue. Additionally, exploring the utilization of large-scale language models may offer a potential solution for capturing keywords within the context of each metric. BERT and MLR complement each other's attributes, and by leveraging their respective strengths, the prediction accuracy of CVSS base vector can be enhanced. Also, regarding the LR method, it is necessary to analyze the input tokens and the vulnerabilities that were incorrect answers.

Furthermore, it is imperative to evaluate the practical effectiveness of each method from the standpoint of security experts. Assessing the potential ramifications of the inaccuracies observed during the evaluation process is crucial.

Acknowledgment

This work was supported by Fund for the Promotion of Joint International Research Grant Number JP21KK0179.

References

[1] National Institute of Standards and Technology (NIST), "NATIONAL VULNERABILITY DATABASE / Understanding Vulnerability Detail Pages," 2022. https://nvd.nist.gov/vuln/vulnerability-detail-pages (accessed 2024-02-28).

[2] Cisco Vulnerability Management, "P2P, vol.1: Analyzing Remediation Strategies," 2018. https://learn-cloudsecurity.cisco.com/kenna-resources/kenna/prioritization-to-prediction-volume-1 (accessed 2024-02-28).

[3] U. K. Singh and C. Joshi, "Quantitative security risk evaluation using cvss metrics by estimation of frequency and maturity of exploit," in *Proceedings of the World Congress on Engineering and Computer Science*, vol. 1, pp. 19–21, 2016.

[4] G. Bhandari, A. Naseer, and L. Moonen, "Cvefixes: automated collection of vulnerabilities and their fixes from open-source software," in *Proceedings of the 17th International Conference on Predictive Models and Data Analytics in Software Engineering*, pp. 30–39, 2021.

[5] Forum of Incident Response and Security Teams, "Common vulnerability scoring system version 3.1: Specification document," 2019. https://www.first.org/cvss/specification-document (accessed 2024-02-28).

[6] C. Elbaz, L. Rilling, and C. Morin, "Fighting n-day vulnerabilities with automated cvss vector prediction at disclosure," in *Proceedings of the 15th International Conference on Availability, Reliability and Security*, pp. 1–10, 2020.

[7] M. R. Shahid and H. Debar, "Cvss-bert: Explainable natural language processing to determine the severity of a computer security vulnerability from its description," in *2021 20th IEEE International Conference on Machine Learning and Applications (ICMLA)*, pp. 1600–1607, IEEE, 2021.

[8] A. Khazaei, M. Ghasemzadeh, and V. Derhami, "An automatic method for cvss score prediction using vulnerabilities description," *Journal of Intelligent & Fuzzy Systems*, vol. 30, no. 1, pp. 89–96, 2016.

[9] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[10] E. Bisong, *Google Colaboratory*. Apress, 2019.

[11] L. Allodi, S. Banescu, H. Femmer, and K. Beckers, "Identifying relevant information cues for vulnerability assessment using cvss," in *Proceedings of the Eighth ACM Conference on Data and Application Security and Privacy*, pp. 119–126, 2018.

[12] C. Molnar, *Interpretable Machine Learning*. 2 ed., 2022.

[13] National Institute of Standards and Technology (NIST), "NATIONAL VULNERABILITY DATABASE / CVE-2018-8569," 2018. https://nvd.nist.gov/vuln/detail/CVE-2018-8569 (accessed 2024-04-30).

[14] J. Han, J. Pei, and M. Kamber, *Data Mining: Concepts and Techniques, 3rd Edition*. Morgan Kaufmann, 2011.

[15] J. C. Costa, T. Roxo, J. B. F. Sequeiros, H. Proenca, and P. R. M. INÁCIO, "Predicting cvss metric via description interpretation," *IEEE Access*, vol. 10, pp. 59125–59134, 2022.

[16] I. Turc, M.-W. Chang, K. Lee, and K. Toutanova, "Well-read students learn better: On the importance of pre-training compact models," *arXiv preprint arXiv:1908.08962*, 2019.

[17] Z. Han, X. Li, Z. Xing, H. Liu, and Z. Feng, "Learning to predict severity of software vulnerability using only vulnerability description," in *2017 IEEE International conference on software maintenance and evolution (ICSME)*, pp. 125–136, IEEE, 2017.

[18] G. C. Moura and J. Heidemann, "Vulnerability disclosure considered stressful," 2023.

Authorized licensed use limited to: University of Education Winneba. Downloaded on September 16,2026 at 12:02:15 UTC from IEEE Xplore. Restrictions apply.